

## 1) စာအုပ် မိတ်ဆက်

### 1.1) ဒီစာအုပ်က ဘာအတွက်လဲ

ဒီစာအုပ် ကတော့ ကျွန်တော် နဲ့ ဒီစာအုပ်ရဲ့ Contributors တွေသိတဲ့ Computer Science နဲ့ ပတ်သက်တဲ့ အကြောင်းတွေကို ရေးထားတာပဲဖြစ်ပါတယ်။ ကျွန်တော်တို့ သိတာတွေရေးတဲ့အတွက် မမှန်ရင်လည်း မမှန်နိုင်သလို မှားလည်းမှားနိုင်ပါတယ်။ အကယ်လို့ အမှားပြင်ချင်တယ် ကိုယ်သိတာလေးတွေ ထည့်ရေးပြီး Contributors (Co-Author) credit လိုချင်ရင်တော့ ဒီစာအုပ် project ရဲ့ Github Repo ကို fork လုပ်ပြီးတော့ဖြစ်ဖြစ် issue ဖွင့်ပြီး အကြံပေးတာဖြစ်ဖြစ် လုပ်လို့ရပါတယ် ။ ကျွန်တော်တို့ရဲ့ Main ရည်ရွယ်ချက်က Computer Science ကိုဘာ Background မှ မရှိတဲ့ သူကနေစပြီး စိတ်ပါဝင်စားတဲ့ ဘယ်သူပဲဖြစ်ဖြစ် ဘာတန်ဖိုးမှ ပေးစရာမလိုပဲ အလကား လေ့လာနိုင်ဖို့ အတွက်ရည်ရွယ်ပါတယ်။

### 1.2) Authors တွေရဲ့ အသိပေးချက်

ဒီစာအုပ်က စစချင်းတော့ အရမ်းပြည့်စုံမှာ မဟုတ်သလို ကိုယ်သိချင်တဲ့ အကြောင်းအရာတွေ ကိုယ်သိပြီးသား အကြောင်းအရာတွေလည်း စစချင်း မပါနိုင်သေးပါဘူး။ ကျွန်တော်လည်း အလုပ်တစ်ဖက် သင်တန်းတစ်ဖက်နဲ့ လုပ်နေရတာကြောင့်ပါ။ ဒါပေမဲ့ လိုတာတွေကို နည်းနည်းစီဖြည့်ဖြည့်ပြီး အဆင်ပြေလောက်တဲ့ အချိန်ကျရင် edition တစ်ခုချင်းစီ ထုတ်ပေးသွားမှာပါ။

### 1.3) License and Code Of Conduct

ဒီ project ရဲ့ license ကိုတော့ WTEPL သုံးထားပါတယ်။ License အရ ဘာမဆိုလုပ်လို့ လို့ရတယ်ဆိုပေမဲ့ Non-Profit Sharing အတွက်ပဲ ခွင့်ပြုပါတယ်။ ပြန်ရောင်းတာမျိုး မလုပ်ဖို့ ခွင့်ပေးပါတယ်။ အကယ်၍ တစ်ယောက်ယောက်က ရောင်းခဲ့ရင်လည်း။ ဒီစာအုပ်ဟာ Online Book PDF file အနေနဲ့ အလကားဖတ်လို့လည်း ရသလို၊ ကိုယ်တိုင် သိသလောက်ဝင်ရေးလို့ရတယ် ဆိုတာ အသိပေးပါရစေ။ ပြန်ရောင်းပေမဲ့ profit ထဲက 25%+ လောက်လိုအပ်တဲ့ နေရာတွေကို ပြန်ပြီး Donate လုပ်ပေးရင် ကောင်းပါမယ်ဗျ။

## 2) Computer Science မိတ်ဆက်

### 2.1) Computer Science ဆိုတာဘာလဲ

**Computer** ဆိုတာက user က ပေးတဲ့ ညွှန်ကြားချက်တွေအတိုင်း အတိအကျလုပ်ဆောင်ပြီး ပြဿနာတွေ ဖြေရှင်းပေးရတဲ့ စက်ပစ္စည်းဖြစ်ပါတယ်။ အသေးစိတ်နဲ့ အမျိုးအစားတွေကိုတော့ **Computer** အတွက်သီးသန့်အပိုင်းမှာ ဆက်ရှင်းပြပေးပါမယ်။ **Science** ဆိုတာကတော့ မြန်မာလိုဆို သိပ္ပံဖြစ်ပြီး အရာရာတိုင်းကို လေ့လာတဲ့ နယ်ပယ်ဖြစ်ပါတယ်။တစ်ကယ်ဆို **Science** နဲ့တင် အဓိပ္ပါယ် ကပြီးပြီ

ဆိုပေမဲ့လည်း တစ်ခါတစ်လေ သူ့နောက်မှာ **Engineering** ပါလာတတ်ပါတယ်။ တစ်ကယ်လည်း ပါ သင့်တယ်လို့ထင်ပါတယ်။ ဘာလို့လဲဆိုတော့ **Engineering** ဆိုတာက **Science** က လေ့လာလို့ ရလာတဲ့ အချက်အလက်တွေကို သုံးပြီး အပြင်လောကရဲ့ ပြဿနာတွေကို ဖြေရှင်းတဲ့ နယ်ပယ်ပါ။ ဆိုတော့ အတိုခေါက် ချုံရရင် **Computer Science Engineering** ဆိုတာ user ရဲ့ ညွှန်ကြားချက်တွေ အတိုင်း အတိအကျ လုပ်ဆောင်ပေးတဲ့ စက်ပစ္စည်းကိုလေ့လာပြီး ပြင်ပ ကမ္ဘာက ပြဿနာတွေကို ဖြေရ ရှင်းပေးရတဲ့ နယ်ပယ် လို့အဓိပ္ပါယ်ရပါတယ်။

### 2.2) Computer ဆိုတာဘာလဲ

**Computer** တွေကနေရာတိုင်းမှာ ရှိနေပါပြီ။ သွားတိုက်တံနဲ့ အိပ်ယာခင်းကစလို့ ဒုံးပျံတွေ ကားတွေ တွေ မှာပါ မရှိမဖြစ်သဘောမျိုး ပါဝင်လာပါတယ်။ ဖုန်းတွေဆိုတာလည်း **Computer** အသေးစားလေးပါ။ **Computer** မှာ **Analog** နဲ့ **Digital** ဆိုပြီး နှစ်မျိုးရှိပါသေးတယ် အဲ့နှစ်မျိုးရဲ့ အသေးစိတ်ကွဲပြားပုံကို ကိုတော့နောက်မှရှင်းပြပါမယ်။ အဓိက ကတော့ **Computer** ဆိုတာ ပေးထားတဲ့ အသေးစိတ် ညွှန်ကြား ကြားချက်တွေအတိုင်း(**Algorithms**) သေချာ အတွက်အချက်လုပ်ပြီး ပြဿနာတွေကို ဖြေရှင်းပေးရတဲ့ စက်လို့ကောက်ချက်ချလို့ရပါတယ်။ **Computer** က အတွက်အချက်တင် လုပ်တာနဲ့တင် **Computer** လို့ခေါ်ပြီလားဆိုတော့ ဟုတ်တယ်လဲ ပြောရသလို မဟုတ်လဲမဟုတ်ပါဘူး။ ကျွန်တော်တို့ ဒီနေ့ ခေတ်အမြင်နဲ့ကြည့်ရင် ဂဏန်းလေးတွေတင် တွက်ချက်ပြီး ပြန်ထုတ်ပေးရတာနဲ့တင် မလုံလောက်တော တော့ပါဘူး အချက်အလက်တွေကို သိမ်းထားနိုင်ရမယ် **Computer** တစ်လုံးနဲ့တစ်လုံး ချိတ်ဆက်ပြီး အချက်အလက်တွေကို ပို့ပေးနိုင်ရမယ်။ လူတွေ ရဲ့ နေ့စဉ် လိုအပ်ချက်တွေကို ဖြည့်စည်းပေးနိုင်တဲ့အပြင် အလုပ်ခွင် သုံးဖြစ်ဖြစ် တစ်ကိုယ်ရည် သုံးပဲဖြစ်ဖြစ် ဘယ်အတွက်သုံးသုံး လွယ်ကူ နေဖို့ အံ့ဝင်ခွင်ကျ ဖြစ်ဖို့ ဆို တစ်ခြားလိုအပ်တဲ့ အချက်အလက်တွေကို သိမ်းဖို့ **Storage** တွေ **Program** တွေအတွက် **Memory** တွေ ချိတ်ဆက်ဖို့ **Network** တွေလို စက်ပိုင်းဆိုင်ရာ ပစ္စည်းတွေ အပြင် အဲ့လို စက်ပိုင်းဆီ ဆိုင်ရာ ပစ္စည်းတွေကို ထိန်းချုပ်ဖို့ စနစ် ပိုင်းဆိုင်ရာ **Software** တွေ လူတွေ အသုံးပြုဖို့ နဲ့ လိုအင်တေ

တွေကိုဖြည့်စည်းပေးနိုင်ဖို့ User ပိုင်းဆိုင်ရာ Software တွေပါလိုပါတယ်။ ဆိုတော့ ကျွန်တော်တို့ ဒီနေ့ခေတ်အမြင်နဲ့ကြည့်ရင်တော့ အဲ့တာတွေအကုန်ပါတဲ့ စက်ပစ္စည်းကို Computer လို့ခေါ်ဝေါ်ကြပါတယ်။

## 2.3) Computer တွေမှာ အမျိုးအစားဘယ်နှခုရှိသလဲ

ကျွန်တော် သိသလောက်တော့ Analog Computer နဲ့ Digital Computer ဆိုပြီး နှစ်မျိုးရှိပါတယ်။ ဒီနေ့ခေတ်မှာတော့ Digital Computer တွေကိုပဲသုံးလာကြပါတယ်။ သူတို့နှစ်ခုအကြောင်းကို သီးသန့်ခွဲပြီး ရှင်းပြပေးပါမယ်။

### 2.3.1) Analog Computer ဆိုတာဘာလဲ

ရှင်းအောင်ပြောရရင်တော့ Analog Computer ဆိုတာ Binary(1, 0) Input တွေအစား physical input တွေ ဥပမာ: လျှပ်စစ် အားတွေတို့ နဲ့ တစ်ခြား ပြင်ပမှာ ရှိတဲ့ အရာဝတ္ထု တွေကို သုံးပြီး ပြဿနာတွေကို ဖြေရှင်းပေးတာပါ။ သူတို့ကိုတော့ ဒီခေတ် modern computer တွေမှာ တွေ့ရမှာမဟုတ်တော့ပါဘူး ဘာလို့လဲဆိုတော့ သူတို့က personal usage အတွက်မကောင်းဘူး ပြင်ပမှာ ရှိတဲ့ physical input တွေကိုသုံးတဲ့အတွက် မြန်ပေမဲ့ Digital လောက် စိတ်မချရသလို မတိကျဘူး အဲ့အတွက် အရေးကြီးတဲ့ အတွက်အချက်တွေအတွက် ဆိုရင် Analog ကို သုံးလို့မရပါဘူး ။ ဒါပေမဲ့ သူ့ရဲ့ အားသာချက်တွေအရဆိုရင် သူက မြန်တယ်, အားသုံးတာလဲနည်းပါတယ်။ ဒီအားနည်းချက်အားသာချက်တွေက သူဟာ physical input အပေါ်မူတည်နေတာကြောင့်ပါ။ ဥပမာ လျှပ်စစ် ကိုပဲ physical input ယူတယ်လို့ ထားလိုက်ပါ။ ဘယ်လောက် volt ဘယ်လောက် အားပေးလဲကစပြီး တစ်ခါတစ်ခါ input ချိန်းတာနဲ့ အရင် အတိုင်းအတိအကျအဖြေပြန်ရဖို့ဆိုတာ မဖြစ်နိုင်သလောက်ကိုဖြစ်ပါတယ် အဲ့တာကြောင့်သူ့ရဲ့ အားနည်းချက် မတိကျတာက ဖြစ်လာတာပါ။ ဒါပေမဲ့ တွက်ချက်မှု တိုင်းက input အားပေးလိုက်တာနဲ့ တန်းပြီး တွက်တော့ realtime မှာ ဖြစ်တဲ့အတွက် အရမ်းမြန်ပါတယ် အဲ့တာကတော့ သူ့ရဲ့အားသာချက်ပါ။ ကမ္ဘာ့ ပထမဆုံး Computer ဖြစ်တဲ့ Analog Computer ကို 1821 မှာ သင်္ချာပညာရှင် Charles Babbage က Design ဆွဲခဲ့ပါတယ်။ သူက စက်အားအပေါ်မှပဲ မူတည်ပြီး Design ဆွဲခဲ့ပြီး Binary(0, 1) အစား deimal(0 9) ကိုသုံးပါတယ်။

### 2.3.2) Digital Computer ဆိုတာဘာလဲ

Digital Computer ကတော့ Analog Computer တွေလို input ကို ပြင်ပ အရာဝတ္ထု တွေကနေယူတာမဟုတ်ပဲ Binary Number လို့ခေါ်တဲ့ 0 နဲ့ 1 ကို input အဖြစ်ယူပြီး Logic Gate တွေကိုသုံးပြီး ပေးတဲ့ အချက်အလက်တွေကို အတွက်ချက်ပြီး ပြဿနာတွေကိုဖြေရှင်းပေးတာပါ။ modern computer အကုန်လုံးလိုလိုက ဒီ Digital Computer တွေပဲ ဖြစ်ပါတယ်။

Logic Gate တွေအကြောင်းကို Cpu အပိုင်းမှာ ရှင်းပြပေးပါမယ်။ သူ့ကို ခွဲပြီးရှင်းပြရရင် အရေးကြီးတဲ့ အပိုင်းသုံးပိုင်းခွဲလို့ရတယ်။ ပထမဆုံးနဲ့ အောက်ဆုံး အပိုင်းကတော့ စက်ပစ္စည်းပိုင်း(Hardware), အဲ့ဒီ Hardware ပိုင်းမှာတော့ စောနကပြောတဲ့ တွက်ချက်တာတွေလုပ်ဖို့ Logic Gate တွေပါတဲ့ Cpu ရယ် အချက်အလက်တွေသိမ်းဖို့ HDD တို့ SSD တို့လို့ Storage device တွေရယ် Program တွေ အသုံးပြုနေစဉ် အချက်အလက်တွေသိမ်းတာနဲ့ တစ်ခြားကိစ္စ တွေလုပ်ဖို့ DDR တို့ DIMM တို့လို RAM(Random Access Memory) ရယ် IO(Input/Output) အတွက် Monitor တို့ Keyboard တို့ နဲ့ Mouse တွေအပြင် တခြား Wifi တို့ bluetooth တို့လည်း ပါဝင်ပါတယ်။ ဒုတိယအနေနဲ့ကတော့ System Software ပိုင်းပါ သူ့မှာတော့ စောနက စက်ပိုင်းဆိုင်ရာ ကိုထိန်းချုပ်ဖို့နဲ့ User ပိုင်းဆိုင်ရာ Software တွေအတွက် စက်ပိုင်းဆိုင်ရာ Hardware ကို management လုပ်ပေးရတဲ့ BIOS တို့, Kernel တို့ နဲ့ Bootloader တို့လို Software တွေပါဝင်ပါတယ်။ တတိယအနေနဲ့ကတော့ User Software ပိုင်းပါ အဲ့တာတွေကတော့ ကျွန်တော်တို့နဲ့ ရင်းနှီးပြီးသား Facebook တို့ Discord တို့နဲ့ တစ်ခြား Software တွေပါဝင်ပါတယ်။ Computer ရဲ့အလုပ်လုပ်ပုံက ကျွန်တော်တို့ လူတွေနဲ့စပ်ပါတယ်။ ကျွန်တော်တို့ ဦးကျောက်က မျက်စိ၊ နား၊ လျှာ စတဲ့ နေရာတွေကနေ အချက်အလက်တွေကိုယူ တွက်ချက်ပြီး ခြေတို့ လက်တို့ ကနေ အချက်အလက်နဲ့ သက်ဆိုင်သလို ပြန်လည် တုံ့ပြန်ပေးတဲ့အပြင်၊ လုပ်ဆောင်နေစဉ် အတွင်းလိုအပ်တဲ့အချက်အလက်တွေကို Short Term Memory ထဲမှာသိမ်းပြီး အရေးကြီးတဲ့အချက်အလက်တွေကိုတော့ Long Term Memory ထဲထည့်ပြီး နောက်သုံးလို့ရအောင်သိမ်းပါတယ်။ Computer ကလဲ အဲ့အတိုင်းပါပဲ Program တွေ Keyboard တွေနဲ့ Mouse တွေလို Input device တွေကနေ အချက်အလက်တွေယူပြီး အတွက်အချက်တွေလ လုပ်ဆောင်ပေးပြီး Monitor တို့ Printer တို့လို Output Device တွေကနေ အချက်အလက်တွေကိုပြန်ပေးတယ်။ အဲ့လိုတွက်ချက်နေတုန်းမှာ ခနတာ မှတ်စရာရှိတာတွေကို RAM ထဲမှာ ခနမှတ်ထားပြီးတော့ အရေးကြီးတဲ့ သိမ်းစရာ file တို့လို အချက်အလက်တွေကို စောနက HDD တို့ SSD တို့လို Storage device တွေထဲမှာ သိမ်းပေးပါတယ်။ ဆိုတော့အဲ့တာကိုကြည့်ရင် Computer နဲ့ လူနဲ့က တော်တော်လေးကိုတူကိုတွေ့နိုင်ပါတယ်။ တစ်ခါတစ်လေမှာ Digital Computer ဆိုတဲ့ အသုံးအနှုန်းအစား Turing Machine လို့ခေါ်လို့လဲရပါတယ်။ Turing Machine အကြောင်းကို ကိုတော့ သက်သက်ရှင်းပြပေးပါမယ်။

## 2.4) Turing Machine နဲ့ Turing Complete ဆိုတာဘာလဲ

Turing Machine ကို 1936 မှာ UK က သင်္ချာ ပညာရှင် Alan Turing ကနေပြီးတော့ တော့ ပထမဆုံး Design ရေးဆွဲခဲ့ပါတယ်။ သူ့ Design မှာ အချက်အလက်တွေပါတဲ့ တိပ်ခွေရယ်၊ ညွှန်ကြားချက်တွေထည့်ဖို့ ကိုယ်ထည် ရယ်၊ အဲ့ဒီ ကိုယ်ထည်ထဲက ညွှန်ကြားချက်တွေအတိုင်း တိပ်ခွေပေါ်က အချက်အလက်တွေကို ဖတ်/ပြင် ဖို့ အရာတစ်ခုရယ်ပါ(ခဲတံဖြစ်ဖြစ် ဘောပင်ဖြစ်ဖြစ်လို့ပဲ

ပဲ မှတ်ကြည့်လိုက်ပါ။)။ သူ့ရဲ့ အလုပ်လုပ်ပုံကလဲ သူ့ရဲ့ Design အတိုင်းပဲ ရိုးရှင်းပါတယ်။ အဲ့ဒီ ခဲတံက တိပ်ပေါ်က အချက်အလက်ကိုဖတ်မယ် ပြီးရင် သူ့ရဲ့ ကိုယ်ထည်ထဲက ညွှန်ကြားချက်ရဲ့ ကဘယ် အခြေအနေမှာ ရှိနေလဲ အဲ့အခြေအနေမှာ ဘာညွှန်ကြားထားသလဲဆိုတာအပေါ်မူတည်ပြီးတော့ ခဲတံက တိပ်ပေါ်က အချက်အလက်ကို ပြင်ရမယ်ဆိုပြင်တယ် မဟုတ်ရင် နောက်အချက်အလက်တစ်ခုစီကိုသွားသွားပါတယ်။ Design ရော အလုပ်လုပ်ပုံရောက ရိုးရှင်းပေမဲ့ သူ့ရဲ့ စွမ်းဆောင်နိုင်စွမ်းက အတွက် အချက်(Compute) လုပ်လို့ရတဲ့ ညွှန်ကြားချက်(Algorithms) မှန်သမျှကို တွက်ဖို့ လုံလောက်တဲ့ အချိန်နဲ့ အချက်အလက်ပေးဖို့ လုံလောက်တဲ့ နေရာရှိရင် အပေါင်း အနုတ်ကစလို့ နာဇီ လျှို့ဝှက် code တွေဖောက်တာ အထိ အခုခေတ် computer တိုင်းလုပ်နိုင်တာအကုန်လုံးကိုလုပ်နိုင်စွမ်းရှိပါတယ်။ တစ်ကယ်လဲ Alan Turing နဲ့ တစ်ခြား သချ် ပညာရှင်တွေပေါင်းပြီး ဒုတိယ ကမ္ဘာစစ်မှာ နာဇီတွေရဲ့ လျှို့ဝှက် code ကို ဖောက်ဖို့ **bombe** လို့ခေါ်တဲ့ Turing Machine ကိုဆောက်ခဲ့ပြီး ဒုတိယ ကမ္ဘာစစ်ကို ခန့်မှန်း ထားတာထက် 2 နှစ်ကနေ 4 နှစ်အထိ တိုအောင်လုပ်ပေးနိုင်ခဲ့ပါတယ်။ စစ်ပွဲအပြီးမှာတော့ Alan Turing နဲ့ John von Neumann တို့ပေါင်းပြီးတော့ **The ENIAC** လို့ခေါ်တဲ့ ကမ္ဘာပထမဦးဆုံး reprogrammable ဖြစ်တဲ့ လျှပ်စစ် Computer ကို Design ဆွဲခဲ့ပါတယ် ။ နောက်တစ်ချက် အနေနဲ့ကတော့ Alan Turing ဟာ Computer တွေလူတွေလို စွမ်းဆောင်နိုင်လား ကိုစမ်းသက်ဖို့အတွက် Turing Test ကိုပါ ရေးသားခဲ့ပါတယ်။ ရှင်းရှင်းပြောရရင်တော့ AI တွေကို စမ်းဖို့အတွက် ဉာဏ်စမ်းသဘောမျိုးပါ။ ဒါ့အတွက် သူ့ကို **Father of Modern Computer and AI** ဆိုပြီး တစ်ချို့ကခေါ်ကြပါတယ်။ Turing Complete ဆိုတာကတော့ရှင်းပါတယ်၊ Turing Machine လုပ်နိုင်သမျှ လုပ်နိုင်တဲ့ အရာကိုခေါ်တာပါ။ ဥပမာဆို Minecraft ရဲ့ Red Stone တို့ complex quantum system တို့ Game Of Life တို့လိုပါ။ Minecraft ရဲ့ Red Stone နဲ့ Minecraft ပြန်ရေးလို့ရသလို Game Of Life နဲ့ AI ဆောက်လို့ရပါတယ်။ ဒါတွေအားလုံးဟာ Alan Turing နဲ့ Turing Completeness Theorem ကြောင်းပါ။ ဒါပေမဲ့ Alan Turing ဟာ သူ့ရဲ့ အတွေးခေါ်တွေပေါ် မူတည်ပြီး တိုးတက်လာမဲ့ ခေတ်ကြီးကို ကြည့်မသွားခဲ့ရရှာပါဘူး။ Alan Turing ကို 1952 မှာ UK ရဲ့ အစိုးရကနေ Gay ဖြစ်တဲ့အတွက် ဟော်မုန်းဆေးတွေအတင်းသောက်ခံခိုင်းခဲ့ရာကနေ 1954 မှာ သူ့ကိုယ်သူ အဆုံးစီရင်ခဲ့ပါတယ်။



Figure 1: Turing Machine ပုံ

### 3) Computer Hardware များအကြောင်း

#### 3.1) မိတ်ဆက်

Computer Hardware ဆိုတာကတော့ Computer တစ်လုံးမှာ အလုပ်လုပ်ဆောင်ဖို့နဲ့ တစ်ခြားလိ လိုအပ်တဲ့ လုပ်ဆောင်ချက်တွေကို CPU နဲ့ တွဲပြီးလုပ်ဆောင်ပေးဖို့ ထည့်သွင်းထားတဲ့ စက်ပစ္စည်း တွေပဲဖြစ်ပါတယ်။ အဲလိုမျိုး Hardware တွေအများကြီးရှိပါတယ်။ ဒါပေမဲ့ ကျွန်တော်သိသလောက် အနည်းငယ်ကိုပဲ ထည့်ပြီးရှင်းပြပေးထားပါမယ်။

#### 3.2) CPU (Central Processing Unit)

CPU ဆိုတာကတော့ Turing Machine ထဲက ခဲတံလေးလိုပါပဲ၊ သူက RAM(တိပ်)ထဲက အချက် အလက် တွေကိုဖတ်ပြီး Software(ကိုယ်ထည်) ကပေးတဲ့ ညွှန်ကြားချက်တွေအတိုင်း RAM ထဲက အချက်အလက်တွေကို ဖတ်/ပြင်ပေးတဲ့ဟာပါ။ သူက Computer Hardware တွေထဲမှာ အရေးကြီးတဲ့ တဲ့ အပိုင်းမှာပါတယ်။ ဥမာဆို လူလိုပဲ လူရဲ့ ခြေ/လက် တို့လို အစိတ်အပိုင်းတွေမပါ ရင်မကောင်းပေမဲ့ အသက်တော့ရှင်နိုင်ပါသေးတယ်။ ဒါပေမဲ့ အချက်အလက်တွေကို တွက်ချက် သိမ်းစည်းထားပေးမဲ့ ဦးကျော ကျောက် မရှိရင် အသက်မရှင်နိုင်ပါဘူး။

##### 3.2.1) CPU မှာဘာတွေပါလဲ ဘယ်လိုအလုပ်လုပ်လဲ

Modern Computer တွေမပေါ်ခင် အရင်ထွက်ခဲ့တဲ့ The ENIAC တို့လို Computer တွေမှာ Vacuum Tube လို့ခေါ်တဲ့ မီးသီးလိုပုံစံ လျှပ်စစ် diode တွေကို သုံးခဲ့ကြပါတယ်။ Vacuum Tube တစ်ခုမှာ Anode, Grid နဲ့ Cathode ဆိုပြီး အပိုင်းသုံးပိုင်းပါပါတယ် အသေးစိတ်ကိုတော့ အောက်မှာ ရှင်းပြပေးပါမယ်။ ကျွန်တော်တို့ LED မီးတွေမပေါ်ခင်က သုံးခဲ့တဲ့ မီးသီးတွေ နဲ့ စင်ပါတယ်။ ဒါပေမဲ့ modern CPU တွေမှာ Vacuum တွေအစား သူတို့နဲ့ လုပ်ဆောင်ပုံချင်းတူတဲ့ transistor တွေကိုပဲသုံးပါတယ် ။ transistor တွေအပြင် CPU မှာ Logic Gate တွေ ယာယီမှတ်ထားဖို့ register တွေ နဲ့ နာရီပါပါတယ်။

##### 3.2.2) Vacuum တွေကဘယ်လိုအလုပ်လုပ်လဲ

Anode ကို အပေါင်းအားပေးထားတဲ့ ဝါယာနဲ့ဆက်ပြီး အဲဝါယာမှာ မီးသီးတစ်လုံးကိုလဲတင်ထားကြည့် ညှိလိုက်ပါ။ Grid ကိုလဲ အပေါင်းအားပေးထားလိုက်ပါ။ သူ့မှာ ပါတဲ့ Cathode ကို လျှပ်စစ် အားပေးလိုက်ပြီးဆိုရင် Cathode က အရမ်းပူလာပြီး electrons တွေထုတ်လွှတ်ပေးပါတယ် ။ အဲဒီ electrons တွေက အနှုတ်အားဖြစ်တဲ့အတွက်ကြောင့် ဆန့်ကျင်ဖက် အားရှိတဲ့ စောနက အပေါင်းအားပေးထားတဲ့ Grid ဆီကို သွားပါတယ်။ ဒါပေမဲ့ တစ်ချို့ electrons တွေက Grid မှာ ပါတဲ့ အပေါက်တွေကနေ လွတ်ထွက်သွားပြီး အပေါင်းအားရှိတဲ့ Anode ဆီကို သွားပါတယ်။ အဲဒီမှာ အပေါင်းအားနဲ့ အနှုတ်အားပေါင်းပြီး လျှပ်စစ် စီးသွားကာ စောနက တင်ထားတဲ့မီးသီးလင်းလာလိမ့်မယ်။

ဒါပေမဲ့ အဲ့တာက Grid က အပေါင်းအားဖြစ်နေတဲ့အတွက်ကြောင့်ပါ။ ကျွန်တော်တို့က အဲ့ဒီအပေါင်းအား အစား Grid ကို အနုတ်အားပေးလိုက်ရင် Cathode ကလာတဲ့ အနုတ်အား electrons တွေက အားတူတဲ့အတွက်ကြောင့် ဆန့်ကျင်ပြီး Anode ဆီကို မရောက်တဲ့အတွက်ကြောင့် မီးလည်းမလင်းတော့ပဲ ပါဘူး။ အဲ့မှာ ကျွန်တော်တို့က မီးလင်းတာကို 1 မီးမလင်းတာကို 0 အဖြစ်ယူလိုက်ရင် 1/0 Binary ရပါပြီ။ ဒါကတော့ အရင်က Computer Cpu တွေအလုပ်လုပ်တဲ့ပုံပါ။

### 3.2.3) Transistor တွေ ကဘာတွေလဲ ဘယ်လိုအလုပ်လုပ်လဲ

Vacuum Tube တွေက အရမ်းအားသုံးလွန်းတာရယ် အရမ်းလေးတာရယ် နဲ့ တစ်ခြား အကြောင်းတေ တွေကြောင့် သာမန် လူတွေသုံးစွဲဖို့ တတ်နိုင်ဖို့ဆိုတာမဖြစ်နိုင်ပါဘူး။ ဒီအတွက် December 23 1947 မှာ Bell Lab ကမ္ဘာပထမဦးဆုံး transistor ကို ထုတ်လုပ်ခဲ့ပါတယ်။ သူ့မှာ အဓိကပါဝင်တာ က သဲကနေရတဲ့ silicon ရယ် ကျောက်တုံးကရတဲ့ phosphorus ရယ် boron ရယ်ပါပါတယ် ။ သူ့ရဲ့ အလုပ်လုပ်ပုံက စောနက Vacuum Tube ပုံစံနဲ့အတူတူပါပဲ။ silicon ရဲ့ အပြင်အခွံမှာ (outermost shell) electron 4 လုံးပါပါတယ် ဒါပေမဲ့ နောက် 4 လုံးပါရင် အရမ်း တည်ငြိမ် (stable) ဖြစ်သွားမှာဖြစ်တဲ့အတွက်ကြောင့် silicon က သူ့ရဲ့ အနီးဆုံးမှာရှိတဲ့ တစ်ခြား silicon 4 ခုနဲ့ ထပ်ပေါင်းတဲ့အခါမှာ အရမ်း stable ဖြစ်တဲ့ crystal lattice ပုံစံ(structure) ဖြစ်သွားကျပါတယ်။ ဒါပေမဲ့ အဲ့လို stable ဖြစ်သွားတာကြောင့် အား(energy) အလုံအလောက် ရတဲ့ တစ်ချို့ electron တွေပဲ အဲ့ဒီ structure ကနေထွက်ပြီး လွတ်လွတ်လပ်လပ် သွားနိုင်ပါတယ် (အိမ်ပြေးတွေ)။ ဒါပေမဲ့အဲ့ ထဲက silicon Atom တစ်လုံးကို စောနက ကျောက်တုံးကရတဲ့ phosphorus Atom တစ်လုံးနဲ့ လဲလိုက်တဲ့အခါမှာ phosphorus ရဲ့ outermost shell မှာ electron 5 လုံးပါတဲ့အတွက်။ စောနကလို stable ဖြစ်တဲ့ structure ရဖို့ ပိုနေတဲ့ electron တစ်လုံးက အပြင်ကို လွတ်ထွက်သွားပြီး စောနကလို stable ဖြစ်တဲ့ structure ပြန်ဖြစ်သွားပါတယ်။ အဲ့ကောင်ကိုတော့ **N-Type** လို့ခေါ်ပါတယ် ဘာလို့လဲဆိုတော့ silicon နဲ့ phosphorus စောနက အချိုးအတိုင်းထပ်ထည့်တဲ့ အခါ structure ကပိုကြီးလာပြီး အနုတ်အား(Negatively charged) ရှိတဲ့ electron တွေလွတ်ထွက်တာ ပိုများလာပါတယ်။ အဲ့ဒီအတွက် အဲ့ကောင်ကနေ အနုတ် လျှပ်စစ်အားထွက်တယ် စောနက Cathode လို့ပါ။ ဒါပေမဲ့ ကျွန်တော်တို့က electron ပိုများတဲ့ phosphorus အစား 3 လုံးပဲရှိတဲ့ boron ကို ထည့်လိုက်တဲ့အခါ သူနဲ့ အနီးမှာရှိတဲ့ electron တွေကို ယူပါတော့တယ် အဲ့ဒီမှာပဲ အပေါင်းအားရှိတဲ့ အပေါက်တွေ ကစပြီး လွတ်ထွက်ပဲ ပါတော့တယ်။ သူ့ကို **P-Type** လို့ခေါ်ပါတယ်။ N-Type အတိုင်းပဲ boron နဲ့ silicon တွေကို အချိုးအတိုင်းထပ်ထည့်တဲ့ အခါ အပေါင်းအားရှိတဲ့ အပေါက်တွေပိုများလာပါတယ်။ Grid နဲ့ နည်းနည်း ကွဲပေမဲ့ သူ့ကို အဲ့တိုင်းမျိုးလို့ပဲမှတ်ထားလို့ရပါတယ်။ transistor မှာ P-Type ရော N-Type ရော နှစ်မျိုးလုံးကို N-Type ကို ဘေးနှစ်ဘက်ကထား P-Type ကို အလယ်မှာထားပြီး N-Type နှစ်ခ

ခုရဲဘေးမှာ လျှပ်စစ်သွားလို့ရအောင် **Source** နဲ့ **Drain** လို့ခေါ်တဲ့ ဝါယာသဘောမျိုးနဲ့ P-Type ရဲ့ အပေါ်မှာ လျှပ်စစ်အားပေးဖို့အတွက် **Gate** လို့ခေါ်တဲ့ ဝါယာသဘောမျိုးပါပါတယ်။ Gate ကနေ လျှပ်စစ်မပေးခင်မှာ N-Type က electron တွေက P-Type မှာရှိတဲ့ အပေါက်တွေထဲရောက်သွားပြီး stable ဖြစ်ကာ နောက်ထပ် Source ကပေးတဲ့ လျှပ်စစ်အားတွေကို အားယူပေးတဲ့ Drain စီမရောက်အောင် တားထားတဲ့ နံရံသဘောမျိုးဖြစ်လာပါတယ်။ အဲ့ကောင်ကို Depletion layer လို့လဲခေါ်ပါတယ်။ ဒါပေမဲ့ Gate ကနေ အပေါင်းအားပေးလိုက်တဲ့အခါ အဲ့ဒီ layer မှာ အပေါက်သဘောမျိုး မျိုး ဖြစ်လာတဲ့အခါ အဲ့အပေါက်ကနေ electron တွေသွား Drain ကိုရောက်တဲ့အခါ လျှပ်စစ်အားက ကိုဖြစ်စေပါတယ်။

### 3.2.4) Logic Gates ဆိုတာဘာလဲ

Logic Gates တွေမှာ အဓိကအနေနဲ့ **NOT Gate**, **AND Gate** နဲ့ **OR Gate** ဆိုပြီး သုံးခုရှိပါတယ်။ နောက်ထပ်အများကြီးထပ်ရှိပါသေးတယ်။ ဒါပေမဲ့ ဒီသုံးခုကိုပဲ ရှင်းပြပေးထားပါမယ်။ Logic Gate အကြောင်းရှင်းအောင်ပြောရရင်တော့ Transistor နဲ့ Resistor တွေကို နေရာအလိုက်ပေးပေါင်းပြီး တစ်ခုရဲ့ output ကို တစ်ခုက input အနေနဲ့ ယူပြီး အလုပ်လုပ်တယ်လို့မှတ်ထားလို့ရတယ်။

AND Gate က input current နှစ်ခုလုံး 1 အပေါင်းအား ဖြစ်နေမှ 1 အပေါင်း output အဖြစ် ပြန်ထွက်ပေးပါတယ်။ တစ်ခုက အနှုတ်အားဖြစ်နေတာတို့ နှစ်ခုလုံးက အနှုတ်အားဖြစ်နေတာတို့ ဆိုရင် အနှုတ်အားပဲပြန်ထွက်ပေးပါတယ်။ AND ကို လုပ်ကြည့်ချင်ရင် ပထမဆုံး Transistor နှစ်ခု A, B ကိုယူ A ရဲ့ Source နဲ့ Gate ကို အားပေးဖို့အတွက် ဝါယာတစ်ခုနဲ့ချိတ် A Drain ကိုတော့ B ရဲ့ Source နဲ့ ချိတ် B ရဲ့ Gate ကို အားပေးဖို့ ဝါယာနဲ့ချိတ်။ ရလဒ်ထွက်မဲ့ B ရဲ့ Drain ကိုတော့ မီးသီးဖြစ်ဖြစ် တစ်ခုခုနဲ့ ချိတ်ကြည့်ထားလိုက်။ A ရဲ့ Source ကို အားပေးလိုက်ပြီး A နဲ့ B ရဲ့ Gate ကို အားမပေးဘဲ နေရင် မီးသီး လုံးဝလင်းလာမှာ မဟုတ်ဘူး ဘာလို့လဲဆိုတဲ့ နှစ်ခုလုံးက 0 ဖြစ်နေတဲ့ အတွက်ကြောင့် လျှပ်စစ်က A ကနေ B ကိုမသွားနိုင်ဘူး။ ဒါဆို A ရဲ့ Gate ကိုပဲ အားပေးရရင်ရမလား။

လုံးဝလင်းလာမှာမဟုတ်ပြန်ပါဘူး ဘာလို့လဲဆိုတော့ A က 1 ဖြစ်ပြီး B က 0 ဖြစ်နေလို့ပါ။ A က အားထုတ်ပေးတယ်။ B မှာလည်း အားဝင်ပေမဲ့ B ကပြန်မထွက်တဲ့အခါမလင်းပြန်ပါဘူး။ အဲ့လိုပဲ B က 1 ဖြစ်ပြီး A က 0 ဖြစ်နေရင်လဲ A ကအားက B ကိုမရောက်တဲ့အတွက်ကြောင့် လင်းမှာမဟုတ်ပါဘူး။ နှစ်ခုလုံး 1 ဖြစ်နေမှသာ A နဲ့ B က နှစ်ခုလုံး အားပြန်ထုတ်ပေးနိုင်လို့လင်းမှာပါ။ OR ဆိုတာကတော့ A ဒါမှမဟုတ် B က အပေါင်းဖြစ်ရင် လျှပ်စစ်ထွက်ပေးတာပါ။ Not ကတော့ ပြောင်းပြန်ပြန်ပေးတာပါ လျှပ်စစ် input ပေးနေရင် သူက ဘာ output မှ ထုတ်မပေးပါဘူး။ input မပေးရင်တော့ output ပေးပါတယ်။ ဥပမာ ပေးရရင်တော့ AND Gate ဆိုတာ နှစ်ဖက်အပြန်အလှန်ရှိတဲ့ချစ်ခြင်း၊ OR Gate က တစ်ဖက်သက်ချစ်ခြင်း၊ Not Gate က ကန့်လန့်တိုက်ချစ်ခြင်းပါ။

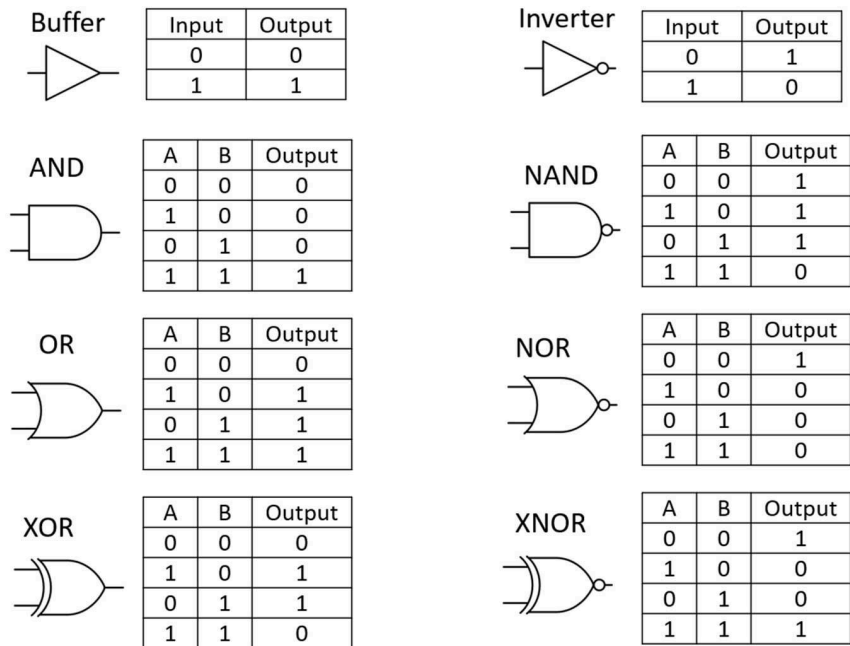


Figure 2: Logic Gates တစ်ချို့ပုံ

### 3.2.5) CPU တွေဘယ်လိုအလုပ်လုပ်လဲ Clock Speed ဆိုတာဘာလဲ

ကျွန်တော်တို့ အစ က CPU တွေက Memory ထဲက Data တွေကို Read Write လုပ်ပြီး အလုပ်လုပ်တာနဲ့ Logic Gate တွေက ဘယ်လိုအလုပ်လုပ်လဲလုပ်လဲ ပြောခဲ့ပြီးပါပြီ။ ဒီတစ်ခါမှတော့တော့ သူတို့နှစ်ခုပေါင်းပြီး ဘယ်လိုအလုပ်လုပ်တာလဲဆိုတာပြောပြပါမယ်။ CPU မှာ Fetch-Decode-Execute လို့ခေါ်တဲ့ FDE ကိုလုပ်ဆောင်ပေးပါတယ်။ စတင်ချင်းမှာ CPU က Program ကပြောထားတဲ့ Memory ရဲ့ အစ ကိုသွားပြီး အဲ့ဒီ Memory Address ကို Fetch လုပ်ကာ သူ့ရဲ့ register ထဲမှာ ခန့်သိမ်းပေးထားပါတယ်။ သိမ်းပြီး သွားတဲ့နောက် Decode အဆင့်ကျ စောနက register ထဲက Memory Address သွားပြီး အဲ့ဒီ Address ထဲမှာ သိမ်းထားတဲ့ Value ဒါမှမဟုတ် လုပ်ဆောင်ရမဲ့ နောက်ထပ် အချက်အလက် ကိုယူပါတယ်။ Execute အဆင့်မှာတော့ အဲ့ဒီနောက်ဆုံး သိမ်းထားတဲ့ register ထဲက အချက်အလက် ဒါမှမဟုတ် ညွှန်ကြားချက်ကို လုပ်ဆောင်ပေးပါတယ်။ အဲ့ကောင်ကိုပဲ Memory Address တစ်ခုချင်းစီ တိုးပြီး လုပ်ဆောင်ပေးသွားတာပါ။ အဲ့ဒါမျိုးကို CPU Clocking ဖြစ်တယ်လို့ခေါ်ပါတယ်။ အဲ့ကောင်တွေက Cpu ထဲမှာ တစ်စက္ကန့်ကို Billion နဲ့ချီပြီးဖြစ်နေတာပါ။ အဲ့ကောင်ကို Hz တို့ GHz တို့လို unit တွေနဲ့တိုင်းပါတယ်။ CPU Clock Rate များလေလေ CPU ကမြန်မြန် တွက်ချက်ပေးနိုင်လေပါပဲ။

### 3.3) Motherboard ဆိုတာဘာလဲ

Motherboard ဆိုတာဘာရယ်မဟုတ်ပါဘူး ကျွန်တော်တို့ မြင်ဖူးနေကျ လျှပ်စစ်ပစ္စည်းတွေမှာ ပါနေက ကျ စိမ်းစိမ်းအပြားကြီး PCB(Printed Circuit Board) အကြီးစား သဘောမျိုးကို Computer

မှာလိုအပ်တဲ့ Hardware တွေချိတ်ဆက်ဖို့အတွက် သီးသန့်ပြင်ဆင်ထားတဲ့ဟာပါ။ သူ့ရဲ့ အဓိကအလုပ်ကတော့ Hardware တွေတစ်ခုနဲ့တစ်ခုကို ချိတ်စက်ပြီး ထိန်းချုပ်ဖို့ပါ။ သူက အချက်အလက်တွေကို ကို မြန်မြန်ဆန်ဆန် လိုတဲ့ Display တို့ Gpu တို့နဲ့ DRAM တို့လိုကောင်တွေကို CPU နဲ့တစ်ခါတည်း ချိတ်ဆက်ပေးပါတယ်။ နောက်ထပ် CPU နဲ့ တစ်ခါတည်း ချိတ်ထားတဲ့ကောင်ကတော့ Chipset ပါ (North Bridge နဲ့ South Bridge လို့ခွဲပြီးတော့လဲပါတတ်ပါတယ်။) သူက အရမ်းမြန်ဖို့မလိုတဲ့ USB device တို့ Storage တို့နဲ့ Speaker တို့တွေ နဲ့ ချိတ်ပေးထားပါတယ်။

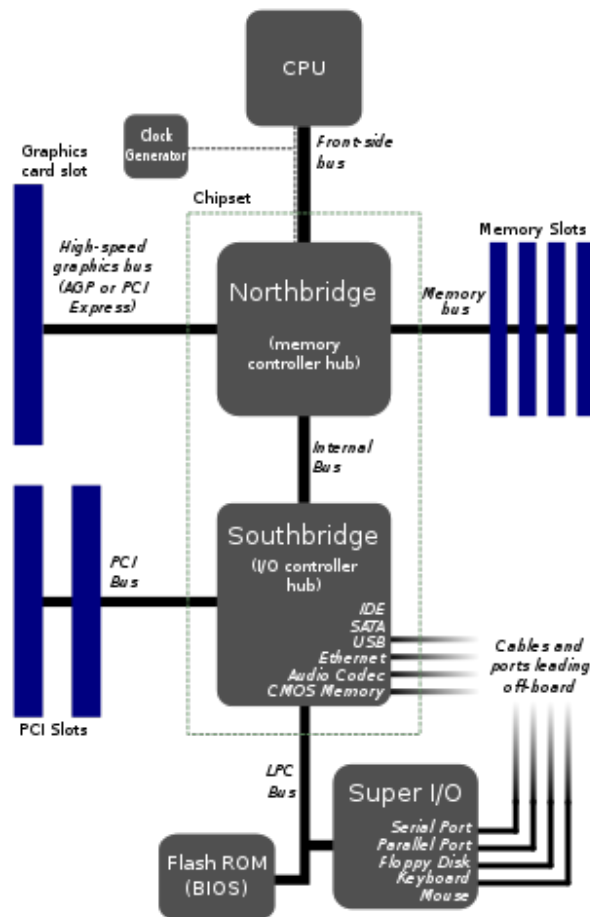


Figure 3: Mother Board Circuit ချိတ်ဆက်ပုံအချို့

### 3.4) RAM (Random Access Memory)

သူကတော့ အများစုနဲ့ ရင်းနှီးပြီးသားဖြစ်မှာပါ။ သူ့ကိုတော့ CPU တွက်ချက်နေတဲ့အချိန်အတွင်း အချက်အလက်တွေ ပေးဖို့ နဲ့ ခနတာသိမ်းစည်းဖို့အတွက် အဓိကသုံးပါတယ်။ CPU က ဘယ်ကလာတဲ့ Data ပဲဖြစ်ဖြစ် SSD တို့ HDD တို့ကနေလာတဲ့ Data ဆိုရင်တောင် RAM ထဲမှာ ထည့်ပြီးမှ process လုပ်ပေးလို့ရတာပါ။ အဲ့တာကြောင့် သူ့ကို Working Memory လို့လဲခေါ်ပါသေးတယ်။ ဒါဆို ဘာလို့ compute လုပ်တဲ့အခါ SSD တို့လို ကောင်တွေမသုံးပဲ RAM ကိုပဲသုံးတာလဲ။ အဖြေကတော့ရှင်းပါတယ် မြန်လို့ပါ နည်းနည်း တင်မြန်တာမဟုတ် ပါဘူး။ အဆပေါင်း 3000 ကျော်လောက်ကိုမြန်ပါတယ်။ ဘာလို့ လို့လဲဆိုတော့ HDD မှာဆို data read/write က လည်နေတဲ့ CD ပေါ်က သံလိုက်စွမ်းအားအတိုင်း လုပ်ဆောင်ပေးတာဖြစ်တဲ့အတွက် အရမ်းနှေးပါတယ်။ SSD က HDD ထက်ဆိုအများကြီးပိုမြန်ပေမဲ့ သူကလဲ 3D ပုံစံ Trillion တွေ့နေချိပြီး Memory Cell တွေ့နဲ့ Terabytes နဲ့ ချိတဲ့ Data တွေကို သိမ်းထားပေးနိုင်ပေမဲ့။ rw(read/write) speed မှာကျ 30 50 microseconds လောက်ကြာပါတယ်။ ဒါပေမဲ့ Ram ကကျ 2D array ပုံစံ Billion နဲ့ ချိတဲ့ Memory Cell တွေ့နဲ့ဖြစ်တာကြောင့် Gb နည်းနည်းလောက်ပဲ ထည့်ထားနိုင်ပေမဲ့ rw speed မှာကျ 1 nanosecond ပဲကြာပါတယ်။ အဆပေါင်း 3000 တောင်ကွာပါတယ်။ ဒါကြောင့် RAM ကိုပဲသုံးတာပါ။

### 3.5) Storage Devices

Storage Device တွေက Data တွေကို အချိန်အကြာကြီး သိမ်းထားတဲ့နေရာမှာသုံးပါတယ်။ Storage Device တွေအများကြီး ရှိပါတယ်။ ဥပမာဆိုရင် Floppy Disk တို့ USB တို့ ကျွန်တော်တို့ အရင်က ဇာတ်ကားခွေတွေထည့်တဲ့ CD ပြားတို့ HDD တို့ SSD (Solid State Drive) တို့ပါ။ သူတို့တွေထဲက အကုန်လုံးလိုလို က လုပ်ဆောင်ပုံခြင်းတူကြပါတယ်။ 1, 0 Data တွေကို electron charge တွေအနေနဲ့ Array ပုံစံ သိမ်းပေးပြီး ပြန်ထုတ်ပေးပါတယ်။ SSD မှာဆို အဲ့လိုသိမ်းပေးတဲ့ Memory cell ကို Charge Trap Flash လို့ခေါ်ပါတယ်။ (ကျွန်တော် ပိုပြီးအသေးစိတ် Data တွေဘယ်လို Memory cell ထဲမှာသိမ်းလည်း။ သူတို့ဘယ်လိုအလုပ်လုပ်လဲဆိုတာကတော့ ဒီ edition မှာ အချိန်မရှိရင် နောက် edition မှထည့်ပေးပါမယ်။)

### 3.6) BIOS(Basic Input Output System)

သူက Motherboard မှာ တစ်ခါတည်းပါလာတတ်ပါတယ်။ သူက Hardware လည်းဟုတ်တယ်လို့ပြောရသလို။ Software လည်းဟုတ်ပါတယ်။ အများစုကတော့ MotherBoard ရဲ့ Firmware(Hardware တစ်ခုခုကို တိုက်ရိုက်ထိန်းချုပ်ပေးတဲ့ Software) လို့လည်းခေါ်ပါတယ်။ သူရဲ့ အဓိကအလုပ်ကတော့ CMOS ထဲက သိမ်းထားတဲ့ Hardware Setting တွေကို ထိန်းချုပ်ပေးပြီး Bootable Drive(Operating System ဒါမှမဟုတ် Bootloader တို့လို Software ထည့်ထားတဲ့ boot လုပ်လို့ရတဲ့ Device) ကို စတင် run ပေးရပါတယ်။ သူက နှေးတာရယ်

Storage များတာတွေကို Handle မလုပ်နိုင်တာရယ်ကြောင့် နောက်ပိုင်း PC တွေမှာ UEFI ကိုသုံးသုံးလာကြပါတယ်။ UEFI က BIOS လိုပဲ ဒါပေမဲ့ ပိုမြန်တဲ့အပြင် Exabytes (1 exa = 1073741824 GB, 1073741 TB) နဲ့ချီတဲ့ partition ပေါင်း 128 ခုကို Handle နိုင်ပါတယ်။ BIOS ကကျ တော့ အများဆုံး 2.2 TB Storage နဲ့ partition 4 ခုကိုပဲ Handle လုပ်ပေးနိုင်ပါတယ်။ BIOS တွေက ဘာမှ မလုပ်ခင် Bootable Device တွေကို မစတင်ပေးခင်မှ Power On Self Test(POST) လို့ခေါ်တဲ့ Test ကို run ပြီး Computer နဲ့ချိတ်ထားတဲ့ Keyboard တွေ Battery တွေ Display တွေအလုပ်လုပ်လားစစ်ပေးပါတယ်။ အကယ်၍ အလုပ်မလုပ်တာတို့ လဲသင့်တာတို့ဆိုရင် ALeart Screen သဘောမျိုးပြပေးပါတယ်။ ကျွန်တော်တို့ Computer အဟောင်းတွေ Bettery မကောင်းတာတွေကိုင်ဖူးရင် သတိထားမိမှာပါ။ Power ဖွင့်လိလိုက်ချင်း Boot တက်မသွားပဲ Your Bettery health is blabla ဆိုပြီး ALeart ပြပါတယ်။ သူက MotherBoard အပေါ်လိုက်ပြီး version တွေကွဲနိုင်ပါတယ်။ သူက MotherBoard တစ်ခုခုဆို တစ်ခုတည်းအတွက်ပဲအလုပ်လုပ်ပြီး တစ်ခြား အမျိုးအစားတွေအတွက်အလုပ်မလုပ်ပါဘူး။ ဒါကြောင့် General ဖြစ်တဲ့ BIOS ဆိုတာလုံးဝကို မဖြစ်နိုင်သလောက်ကိုဖြစ်ပါတယ်။

## 4) System Software များအကြောင်း

### 4.1) မိတ်ဆက်

System Software ဆိုတာကတော့ Hardware ကို တိုက်ရိုက် control လုပ်ပေးတဲ့ Software တွေပါ။ ကျွန်တော်တို့ သုံးနေကျ Discord တို့လို Facebook တို့လို Code တွေနဲ့ ရေးထားတာပါပဲ ဒါပေမဲ့ သူတို့က Hardware တွေကို တိုက်ရိုက်ထိန်းချုပ်ပေးပြီး အဲ့ဒီ Facebook တို့ Discord တို့လို User Software တွေ Hardware ကိုသုံးဖို့အတွက် API ပြန်ထုတ်ပေးရပါတယ်။ သူ့မှာ အဓိကအာ အားဖြင့်တော့ BIOS, Bootloader, Kernel, Kernel Drivers & Modules တွေပါဝင်ပါတယ်။ BIOS အကြောင်းကိုတော့ ကျွန်တော်တို့ ရှင်းပြပြီးပါပြီ။ ကျန်တာတွေကို ဆက်ရှင်းပြပေးပါမယ်။

### 4.2) Bootloader

Bootloader ဆိုတာ BIOS ပြီးရင် ဒုတိယ run ပေးတဲ့ Software ပါ BIOS ကနေ POST run ပြီးရင် Bootloader ရှိတဲ့ MBR ဆိုတဲ့ partition မှာ သွားရှာပြီး run ပေးရပါတယ်။ ဒါပေမဲ့ UEFI မှာတော့ Bootloader file ထည့်ဖို့ partition သက်သက်ဆောက်ပေးရပါတယ်။ သူကနေမှ Operating System တွေရဲ့ Kernel File တွေရှိတဲ့ Drive Location ကို ရှာပေးပြီး User ကြိုက်တဲ့ OS ကို boot နိုင်အောင်ပြပေးရပါတယ်။ သူက Physical Memory ကိုလဲ Map လုပ်ပေးရပြီး Virtual Memory address တွေပြန်ထုတ်ပေးရပါတယ်။ user ကြိုက်တဲ့ OS ကိုရွေးပြီး သွားပြီဆိုရင် အဲ့ OS နဲ့ဆိုင်တဲ့ Kernel code file ကို Memory ထဲမှာ run ပေးပြီး Map လုပ်ထားတဲ့ Memory တွေကို control လုပ်ဖို့ပေးလိုက်ရပါတယ်။ bootloader code ကို ကြည့်ချင်ရင် ကျွန်တော့ [Github Repo](#) မှာ ကြည့်ကြည့်လို့ရပါတယ်။

### 4.3) Operating System (OS)

ပထမဆုံးအနေနဲ့ကတော့ OS ဆိုတာ Operating System ပါ Online Shop မဟုတ်ပါဘူး။ သူက BIOS ကလွဲလို့ System Software တွေအကုန်လုံး စုပေါင်းပြီး User Software တွေအတွက် Syscall လို Api တွေပြန်ထုတ်ပေးဖို့လိုတဲ့ အရာတွေအကုန် ပေါင်းပြီး OS လို့ခေါ်ပါတယ်။ သူ့မှာ အဓိကပါတာကတော့ Kernel, Kernel Driver တွေ Kernel Modules တွေ Bootloader တို့ပါပါတယ်။ OS တွေအများကြီးရှိပါတယ်။ ကိုယ်တိုင်လုပ်ချင်ရင်တောင်အရမ်းကြီးမခတ်ပဲလုပ်ရပါတယ်။ ကျွန်တော် ဒီထဲမှာ တစ်ခုရေးပြဖို့လည်းရှိပါတယ်။ နောက် edition ကျရင်ပေါ့။ အဲ့ထဲကမှ နာမည်ကြီးကြီးတဲ့ တစ်ချို့ OS တွေက Windows, Mac, Gnu/Linux, BSD, RTOS တို့လိုကောင်မျိုးတွေပေါ့။ User Software တွေက OS တစ်ခုနဲ့ တစ်ခုမှာ ကွဲပါတယ်။ Windows User Software (.exe file, ex. Discord.exe Windows file) တွေကို Gnu/Linux တို့ Mac တို့မှာ Native run မရပါဘူး။ ဘာလို့လဲဆိုတော့ OS တစ်ခုနဲ့တစ်ခု Hardware တွေ Handle လုပ်ပုံ

ပုံခြင်း Program File Header တွေ Syscall Api တွေထုတ်ပေးပုံခြင်းမတူလို့ပါ။ ရှင်းရှင်းပြောရရင် OS တစ်ခုနဲ့ တစ်ခု အကုန်လုံးလိုလိုကွဲပါတယ်။

#### 4.4) Kernel

Kernel ဆိုတာကတော့ Software ရဲ့ CPU သဘောပါပဲ သူမပါပဲ ဘယ် program မှ run လို့ မရဘူးလို့ကိုပြောရပါတယ်။ ဘာလို့လဲဆိုတော့ သူကနေမှ Hardware တွေကို Software တွေသုံးလို့ လို့ရအောင် Map လုပ်ပြီး API တွေပြန် ပေးရတာပါ။ ကိုယ်တိုင် လိုတဲ့ Hardware Driver တွေ Memory Map လုပ်တာတွေကို မလုပ်တတ်ရင်တော့ Kernel မပါပဲ ဘာ Software မှရေးမရပါဘူး။ သူက ပထမဆုံး ဘာမှမလုပ်ခင် Device တွေကို စစ်ပြီး လိုအပ်တဲ့ Device Driver တွေ Kernel Modules တွေကို စတင်ပေးပါတယ်။ ဥပမာ Display အတွက် VGA Driver တို့ GPU Driver တို့နဲ့ Binder Modules တို့လို့ပါ။ ဒါတွေကို စပေးပြီးပြီးဆိုရင်တော့ သူက Systemd တို့ Openrc တို့လို Init process ကိုစတင်ပေးပါတယ်။ သူက ဘာလုပ်ပေးရတာလဲဆိုတော့ အများကြီးပါပဲ User Config လုပ်ထားတဲ့ Background process တို့နဲ့ လိုအပ်တဲ့ Wifi တို့ Bluetooth တို့လို Service တွေကို စတင်ပေး ရပါတယ်။ အဲ့ဒါအပြင် မတူတဲ့ File System တွေတို့ Disk တို့ကို ကို သုံးလိုရအောင်လုပ်ပေးရပါတယ်။ နောက်ပြီး User Login ဝင်တာတို့ကိုလည်း Handle လုပ်ပေး ရပါတယ်။ အရှင်းဆုံးမှတ်ထားရရင်တော့ Init System တွေက Kernel က လိုတဲ့ Driver တွေ Module တွေစတင်ပေးပြီးရင် ကျန်တဲ့ စပေးဖို့ လိုအပ်တဲ့ဟာတွေကို စပေးတဲ့ဟာလို့မှတ်ထားလို့ရပါတယ် ။ Kernel ရဲ့ အလုပ်ကအဲ့မှာတင်ပြီး သွားတာမဟုတ်ပဲ အများကြီးကျန်ပါသေးတယ်။ သူက ကျွန်တော်တို့ User Software တွေ Hardware ကိုသုံးဖို့ Api တွေပြန်ထုတ်ပေးရပါတယ်။ အဲ့ Program run လိုက်တဲ့အချိန်မှာလဲ သူရဲ့ Header က မှန်ရဲ့လား ဘယ် Syscall တွေသုံးထားလဲ။ ဘယ် Syscall ဆို ဘယ် Driver ကိုသုံးရမှာလဲ။ Thread တွေထပ်ထုတ်ပေးရမလား ဒီ Data တွေကို ဘယ် Memory Address မှာ သွားသိမ်းပေးရမလဲ ဒီလို အောက်ခြေသိမ်း အလုပ်တွေအကုန် သူမှာတာဝန်ရှိ ရှိပါတယ်။ Kernel မှာ Monolithic Kernel, MicroKernel နဲ့ တစ်ခြားအမျိုးအစားတွေရှိ ရှိပါသေးတယ်။ သူတို့အပေါ်မူတည်ပြီး Kernel က ဘယ်နေရာမှာကောင်းတယ်။ ဘယ်လို handle လုပ်တယ်။ ဒါတွေကွဲပါသေးတယ်။ အဲ့ဒီလို အသေးစိတ်တွေကိုတော့ နောက် edition မှာထည့်ပေးပ ပါမယ်။ အရမ်း Simple ဖြစ်တဲ့ Rust နဲ့ ရေးထားတဲ့ Kernel နဲ့ VGA Device Driver ကို ကျွန်တော့်ရဲ့ [Github Repo](#) မှာ ကြည့်ကြည့်လို့ရပါတယ်။

#### 4.5) Device Driver

Device Driver လို့လဲခေါ်တဲ့ Kernel Driver တွေကို Kernel က Hardware တွေကို handle လုပ်ဖို့ အတွက်သုံးပါတယ်။ သူတို့တွေက Kernel အပေါ်မူတည်ပြီး ပါတဲ့ပုံနဲ့ လုပ်ဆောင်ပ ပုံတွေကွာပါတယ်။ ဥပမာ Linux တို့လို Monolithic Kernel မျိုးမှာဆိုရင် Kernel Driver

တွေက Kernel code ထဲမှာ Kernel ထဲမှာ တစ်ခါတည်းပါပြီးသားပါ manual လဲ install လို့ ရပါတယ်။ ဒါ့အတွက်ကြောင့် Linux ကအရမ်းမြန်ပါတယ်။ ဒါပေမဲ့ Windows တို့လို MicroKernel တွေမှာတော့ တစ်ခါတည်းမပါပဲ ကိုယ်တိုင် install ရပါတယ်။ သူတို့တွေက Kernel ပြီးရင် အရမ်း အရေးပါတဲ့ System Software တွေဖြစ်ပါတယ်။ သူတို့မပါရင် ဘယ် Hardware ကိုမှ သုံးလို့ရမှာ မဟုတ်ပါဘူး။ တစ်ခါတစ်လေမှာ ကိုယ် Graphic card ပြောင်းလိုက်တာတို့ ကိုယ့် Pc ထဲကနေ တစ်ခုခုပြောင်းလိုက်တာတို့ဆိုရင် Windows မှာချက်ချင်းသုံးမရပဲ Linux မှာကျ ရနေတာတွေရှိပါလိမ့် မယ်။ အဲ့တာ သူတို့ မတူတဲ့ Kernel နှစ်ခု ကြောင့်ပါ စောနကပြောသလို Windows မှာ Device တစ်ခုခုပြောင်းလိုက်လို့ မရရင် အဲ့တာ Driver မရှိလို့ပါ။ Linux မှာကျ လိုတဲ့ Driver တွေအကုန်ပဲ ပါပြီး သားဆိုတော့ ချက်ချင်းရပါတယ်။ FOSS (Free And Open Source Software) မဟုတ်တဲ့ Driver တစ်ချို့လောက်ကိုသာ ကိုယ်တိုင်သွင်းရင်းရင်းရမှာပါ။

#### 4.6) Shell

Shell ဆို တာကတော့ User Software နဲ့ System Software နှစ်ခုကြားမှရှိတဲ့ကောင်လို့ပြောလို့ ရပါမယ်။ ဥပမာပြောရရင် Bios လိုပါပဲ။ သူက Os ကို User တွေသုံးလို့ရအောင် CLI (Command Line Interface) တို့လိုမျိုးနဲ့ program run တာတွေ program တွေကို Script လုပ်ပြီး run တာတွေဒီလိုမျိုးတွေကို လုပ်ဆောင်ပေးပါတယ်။ သူက User Software တစ်ခုပါပဲ ဒါပေမဲ့ သူက တစ်ခြား User Software တွေကို Os နဲ့ ချိတ်ဆက်ပြီး အလုပ်လုပ်ပေးဖို့ Layer သဘောမျိုးထားပေး ပေးထားတာပါ။ အဲ့ဒီအတွက် သူမှာ I/O pipeline တွေ Program run တာတွေ File Manage လုပ်တာတွေအတွက် Build-In Command တွေပါပါတယ်။ အခုခေတ် Modern Shell တွေမှာ ဆိုရင် သူတို့နဲ့ ချိတ်ဆက်ဖို့ Customize လုပ်ဖို့ plugin တွေပါတည့်လို့ရတဲ့အပြင် Scripting Language အနေနဲ့ပါသုံးလို့ရပါတယ်။ Scripting နဲ့ Programming Language ကွာတာက Programming က သူ့ရဲ့ Library တွေကိုသုံးပေမဲ့ Scripting က တစ်ခြား Program တွေကို Library သဘောမျိုးသုံးပါတယ်။ Shell တွေအများကြီး ရှိပါတယ်။ Linux နဲ့ Mac တို့လို Unix base Os တွေမှာဆိုရင် Bash Shell ကို Default အနေနဲ့ ပါလာတတ်ပါတယ်။ Windows မှာ ဆို Powershell တို့ Ms-Dos တို့ရှိပါတယ်။ ဒါ့အပြင် Linux တို့လို unix base တွေအတွက် nushell, zsh, fish ဆိုပြီး အများကြီးရှိပါသေးတယ်။

## 5) User Software များအကြောင်း

### 5.1) မိတ်ဆက်

User Software ဆိုတာကတော့ ကျွန်တော်တို့ နေ့စဉ်သုံးနေတဲ့ Facebook တို့ Discord တို့လို လို Software တွေကို ပြောတာပါ။ ဒီထဲက အများစုက 0s ကို ထိန်းချုပ်ပေးတာ 0s ကပေးတဲ့ တဲ့ Api တွေကို Programming Language သုံးပြီး ရေးထားတဲ့ Program တွေပါပါတယ်။ Hello World Program လေးကစလို့ Telegram တို့ Youtube တွေထိ အပါအဝင် User Software တွေ Billion နဲ့ ချီပြီး ရှိပါတယ်။ ဒီထဲမှာ Facebook တို့လို Youtube တို့လိုကောင်တွေပါမှာမ မှန်း အများစုသိကြပါတယ်။ ဒါပေမဲ့ အများစုမသိတဲ့ 0s ကို ထိန်းချုပ်ပေးတဲ့ 0s က Api တွေကို သုံးပြီး တစ်ခြား User Software တွေပိုပြီး လွယ်လွယ်ကူကူသုံးရအောင် ထပ်ပြီး Api ထုတ်ပေးတဲ့ System Level User Software တွေကိုပဲ ပြောပေးသွားပါမယ်။

### 5.2) Display Server

ကျွန်တော်တို့ သာမန် computer သုံးတဲ့ သူတွေ computer ဖွင့်လိုက်ရင် ကျွန်တော်တို့ Mouse တွေကိုင်ပြီး သုံးလို့ရမဲ့ Firefox တို့လို Application Window တွေပေါ်လာဖို့အတွက် Graphical User Interface လို့ခေါ်တဲ့ GUI ကိုတွေ့ရမှာပါ။ ဒါပေမဲ့ သူက ရိုးရှင်းမနေပါဘူး ကျွန်တော်တို့ Gui တွေမပေါ်ခင် Cli ဆိုတဲ့ Command Line Interface လို့ခေါ်တဲ့ Shell Command တွေ သုံးလို့ပဲရခဲ့တာပါ သူ့မှာ ဘာ Graphic မှမပါပါဘူး။ ဒါပေမဲ့ နောက်ပိုင်းမှာတော တော့ Display Server တွေပေါ်လာပါတယ်။ သူတို့က ဘာလုပ်ပေးတာလဲဆိုတော့ ဒီ Computer Screen တို့ Monitor, Keyboard နဲ့ Mouse တို့လို ကောင်တွေရဲ့ အပြောင်းအလဲကို ယူပြီး ဘာတွေပြောင်းလဲသွားတယ် Mouse ဆိုရင်လဲ User ကဘယ် position မှာ ဘာလုပ်လိုက်တယ် ဆိုတာကို Program လုပ်ပြီး လုပ်ချင်တာလုပ်ဖို့ Api ပြန်ထုတ်ပေးရတဲ့ကောင်တွေပါ အဲ့ဒီ Api တွေကို Programming Language တစ်ခုနဲ့သုံးပြီး Window Manager လို့ခေါ်တဲ့ ကျွန်တော်တို့ လက်ရှိသုံးနေတဲ့ GUI ကြီး ကို ဖြစ်လာအောင် ထောက်ပံ့ပေးရပါတယ်။ သူ့မှာဆိုရင် Xorg တို့ Wayland တို့ဆို ဆိုပြီးအမျိုးမျိုးရှိပါတယ်။ Windows မှာဆိုရင်တော့ WDDM (Windows Display Driver Model) ကိုသုံးပါတယ်။

### 5.3) Window Manager and Desktop Environment

စောနကပြောခဲ့သလိုပါပဲ Window Manager ဆိုတာ Display Server က ပြန်ထုတ်ပေးတဲ့ Api တွေကို သုံးပြီး Developer စိတ်တိုင်းကျ Developer ကြိုက်တဲ့ Design philosophy အတိုင်း တစ်ခြား User Software တွေရဲ့ window တွေကို Keyboard နဲ့ Mouse ကဖြစ်သွားတဲ့ Event တွေအတိုင်း handle လုပ်ပေးရတာပါ။ ဥပမာဆိုရင် Application Window အသေးအကျယ် ချုံ့ချဲ့

လုပ်တာတို့ Layout Style ချိန်းတာတို့ စာရိုက်တာတို့ နဲ့တစ်ခြားဟာတွေအများကြီးပါ။ သူတို့တွေကလည်း အများကြီးရှိပါတယ်။ Kde မှာဆို Kwin, Xfce ရဲ့ Xfwm, Xorg မှာဆိုရင် i3, bspwm, dwm နဲ့ Wayland အတွက်ဆို Hyprland တို့ river တို့လို မျိုးအများကြီး ရှိပါတယ်။ window manager တွေက ပေါ့ပါးတယ်။ Customizable အရမ်းဖြစ်တာကြောင့် Linux Customization အတွက် နာမည်ကြီးပါတယ်။ Desktop Environment ဆိုတာကတော့ Window Manager ကိုမှ additional User Software တွေတွဲပြီးပါတာကိုပြောတာပါ။ ဥပမာဆိုရင် Window Manager မှာ Setting App တို့ ဘာတို့မပါပါဘူး။ ကိုယ်တိုင် Config File ထဲကနေ ဝင်ပြင်တာတို့လိလိုမျိုးလုပ်ရပါတယ်။ ဒါပေမဲ့ Desktop Environment မှာကျ အဲ့လိုမျိုးတွေတစ်ခါတည်းပါတဲ့အပြင် Movie ကြည့်တဲ့ App သီချင်းနားထောင်တဲ့ App ဒါတွေအကုန် ပါလာပါတယ်။ အဲ့ဒီအတွက်ကြောင့် သာမန် user တွေအတွက်ကောင်းပေမဲ့၊ စက်သိပ္ပံမကောင်းတဲ့သူတွေ အရမ်းများတာမလိုချင်ပဲ minimal ပဲကြိုက် တဲ့သူတွေအတွက်ကျ မလိုတာတွေအများကြီး ပါတဲ့အတွက် အရမ်း bloat ဖြစ်ပါတယ်။ Desktop Environment တွေလည်း အများကြီး ရှိပါတယ်။ Kde တို့ Gnome တို့ Xfce တို့လိုပါ။ Simple ဖြစ်တဲ့ Window Manager ကို rust မှာ ဘယ်လိုရေးရလဲဆိုတာကို ကျွန်တောတော့ရဲ့ SSWM (Saffron Spring Window Manager) Github Repo လေးမှာ ဝင်ကြည့်လို့ရပါတယ်။

## **6) တစ်ခြား Computer Science နဲ့ ပတ်သက်တဲ့ အကြောင်းအရာများ**

### **6.1) Networking အကြောင်း**

#### **6.1.1) Computer နဲ့ Networking ဆိုတာဘာလဲ**

#### **6.1.2) မတူညီတဲ့ Networks များ**

#### **6.1.3) Ipv4 နဲ့ Ipv6**

## **6.2) Multi Media**

**6.2.1) Computer နဲ့ Multi Media ဆိုတာဘာနုဉ်းတာလဲ**

**6.2.2) Media File နုဉ်းဘာလဲ**

6.3) နည်းပညာပိုင်းဆိုင်ရာ ကိုယ်ရေးအချက်အလက်တွေ့ နဲ့ နည်းပညာ လုံခြုံရေး အကြောင်း

6.3.1) နည်းပညာပိုင်းဆိုင်ရာ ကိုယ်ရေးအချက်အလက်ဆိုတာဘာလဲ အရေးကြီးလား

6.3.2) နည်းပညာလုံခြုံရေး အကြောင်း

## **7) Computer တွေကို ကိုယ်လိုရာ ခိုင်းစေချင်း (Programming)**

**7.1) Programming ဆိုတာဘာလဲ**

**7.2) Programming Language တွေကဘာတွေလဲ**

**7.3) Programming Language အမျိုးအစားတွေနဲ့အကြံပေးချက်များ**

#### **7.4) Python Programming**

Python Programming လေ့လာဖို့အတွက် Resource လေးပါ

#### **7.5) C Programming**

C Programming လေ့လာဖို့ Resource လေးပါ

#### **7.6) Rust Programming**

ဒါကတော့ ကျွန်တော့်ရဲ့ မြန်မာလို Rust Course လေးပါ

## 8) စာအုပ်လေး ပိုပြည့်စုံအောင် ဖြည့်စည်းပေးကြတဲ့ Contributors များ

- Linus Walker [[Facebook Acc](#), [Github Acc](#), [Mail](#)]